

Instituto Superior Politécnico de Tecnologias e Ciências (ISPTEC)
Departamento de Engenharia e Tecnologias
Licenciatura em Engenharia Informática · 2025/2026

BASE DE DADOS II

Lab #02 — Exploração de Banco de Dados Distribuídos

Exploração de um Banco de Dados Distribuído · MercadoKwanza

Identificação do Trabalho

Campo	Descrição
Grupo	Grupo 1
Cenário / Situação	Cenário A — Defensor na primeira ronda (Situação A)
Problema	Problema 1 — Replicação + Fragmentação
Turma	M1
Data	21 de Junho de 2026
Docente	Judson Paiva — judson.paiva@isptec.co.ao

Elementos do Grupo

Nome	Nome
Nuno Mendes	Khelv Costa
Rafaela Mendes	Paula Alexandre

Entrega: [BDD II] Lab02 — Grupo 1 — Situação A
Destinatário: judson.paiva@isptec.co.ao

Índice

1. Capa
2. Parte 1 — Resultados (Fases 0 a 4)
3. Parte 1 — Reflexões
4. Registo de Uso de IA
5. Parte 2 — Implementação (Cenário A)
6. Parte 2 — Análise
7. Parte 2 — Reflexões
8. Dificuldades
9. Conclusão
10. GitHub
11. Referências (APA 7.^a edição)

2. Parte 1 — Resultados (Fases 0 a 4)

Fase 0 — Preparação do Ambiente (Docker)

Subimos o cluster com docker compose up -d, que arranca quatro contentores: três nós MySQL 8.0 (Luanda, Benguela e Huambo) e um nó MongoDB 7.0. Confirmámos com docker ps que todos ficaram no estado healthy. Cada nó MySQL tem um server-id único (requisito obrigatório para a replicação) e a versão foi confirmada em todos: MySQL 8.0.46.

Contentor	Imagem	Estado	Portas	server-id
no-luanda	mysql:8.0	Up (healthy)	3301→3306	1
no-benguela	mysql:8.0	Up (healthy)	3307→3306	2
no-huambo	mysql:8.0	Up (healthy)	3308→3306	3
mongo-kwanza	mongo:7.0	Up (healthy)	27017→27017	—

Registo de Resultados — Fase 0

Item	Valor
N.º de linhas em VENDA após importação	30 001
N.º de linhas em ITEM_VENDA	89 933
N.º de clientes importados	5 000
N.º de produtos importados	203 (base)
Erros encontrados no script da IA (s/n — quantos?)	Sim — 2 (ver Secção 8)

Dificuldade nesta fase: ao ligar com -pkwanza2024 recebíamos ERROR 1045 (28000): Access denied, apesar de o contentor estar healthy. Percebemos que o healthcheck (mysqladmin ping) devolve sucesso mesmo com password errada e que o volume Docker antigo tinha mantido a password da primeira inicialização. Resolvemos com docker compose down -v && docker compose up -d para recriar os volumes de raiz.

Fase 1 — Criação do Schema mercadokwanza

O schema foi criado automaticamente a partir do ficheiro dados/mercadokwanza_p1.sql, montado em /docker-entrypoint-initdb.d. Ficou com 7 tabelas relacionais, ligadas por chaves estrangeiras:

Tabela	Descrição
PROVINCIA	Províncias de Angola (Luanda, Benguela, Huambo)
LOJA	Lojas, cada uma ligada a uma província (provincia_id)
CLIENTE	Clientes do MercadoKwanza
PRODUTO	Catálogo de produtos (descrição, categoria, preço)
STOCK	Quantidade de cada produto em cada loja

Tabela	Descrição
VENDA	Cabeçalho da venda (loja, cliente, data, total)
ITEM_VENDA	Linhas de cada venda (produto, qtd, preço, desconto)

Registo de Resultados — Fase 1

Item	Valor
N.º de contentores com STATUS Up	4
Tempo de arranque do cluster (aprox.)	~25 s (1.ª vez com download das imagens)
COUNT(*) FROM VENDA (Luanda)	30 001
COUNT(*) FROM CLIENTE (Luanda)	5 000
Os dados do SQL foram carregados automaticamente?	Sim (S)

Fase 2 — Carga de Dados Inicial

O mesmo dataset carregou os dados base. Confirmámos as contagens em Luanda logo após a carga. O dataset é determinístico (mesmos valores em qualquer máquina), o que torna os resultados reproduzíveis por qualquer elemento do grupo.

Registo de Resultados — Fase 2 (Contagens)

Tabela	Registos
PROVINCIA	3
LOJA	15
CLIENTE	5 000
PRODUTO	203 (base; passou a 253 após o P1-A)
STOCK	3 000
VENDA	30 001 (base; +1 na Fase 4-B)
ITEM_VENDA	89 933

Fase 3 — Verificação de Integridade e Distribuição Geográfica

Validámos que as 15 lojas estão distribuídas pelas 3 províncias (5 lojas por província) e que cada loja tem o seu próprio stock. Esta distribuição geográfica é precisamente o que torna possível, mais à frente, a fragmentação horizontal de STOCK por província (Parte 2). Não foram detetadas violações de integridade referencial.

Registo de Resultados — Fase 3 (Distribuição)

Verificação	Resultado
Lojas por província	5 + 5 + 5 = 15
Cada loja tem stock próprio?	Sim (S)

Verificação	Resultado
Violações de integridade referencial	0 (nenhuma)

Fase 4-B — Transação Distribuída (2PC Simplificado) · [scripts/transacao.py](#)

Cenário testado: cliente de Luanda compra um produto cujo stock está em Benguela. O stock é decrementado em Benguela (porta 3307) e a venda é registada em Luanda (porta 3301), com commit coordenado nos dois nós. Em caso de erro, rollback em ambos.

Registo de Resultados — Fase 4-B

Item	Valor
Stock em Benguela ANTES da transação	153
Stock em Benguela APÓS a transação (sucesso)	143 (-10) ✓
N.º de vendas em Luanda ANTES	30 001
N.º de vendas em Luanda APÓS (sucesso)	30 002 (+1) ✓
Resultado com erro forçado — ROLLBACK executado?	Sim (S)
Stock em Benguela após ROLLBACK (igual ao inicial?)	143 — Sim, igual ✓

Log da execução com sucesso (passo 21):

```
[Benguela] Stock decrementado: -10 unidades
[Luanda] Venda registada: id=30003
✓ Transação concluída com sucesso em ambos os nós.
```

Log da execução com erro forçado (passo 22 — INSERT da VENDA em Luanda comentado):

```
[Benguela] Stock decrementado: -10 unidades
X ROLLBACK efectuado em ambos os nós.
Motivo: 1048 (23000): Column 'venda_id' cannot be null
```

Verificação de consistência após ROLLBACK (passo 23): o stock em Benguela voltou a 143 e o número de vendas em Luanda manteve-se em 30 002 — nenhum nó ficou com alterações parciais, o que confirma a atomicidade distribuída.

3. Parte 1 — Reflexões (Fases 0 a 4)

Sobre a Fase 4-B (transação distribuída)

A Fase 4-B foi, para o nosso grupo, o momento em que o conceito de “transação distribuída” deixou de ser apenas teoria e passou a fazer sentido na prática. Estávamos a mexer em dois servidores MySQL diferentes ao mesmo tempo — o stock em Benguela e a venda em Luanda — e a grande questão era: como garantir que ou as duas operações acontecem, ou nenhuma acontece?

No caso de sucesso, percebemos que a transação só ficou definitivamente gravada depois de as duas fases (decremento do stock e registo da venda) terem corrido sem qualquer erro e de termos chamado o `commit()` nas duas ligações. Isto ilustrou de forma muito clara a propriedade de atomicidade: o princípio do “tudo ou nada”. Enquanto não houvesse `commit`, nada estava realmente seguro.

O caso do erro forçado foi o mais interessante de observar. Ao comentar o `INSERT` da venda em Luanda, provocámos uma falha de propósito, e o programa entrou no bloco de tratamento de erro, executando `rollback()` nas duas ligações. Um pormenor que nos chamou a atenção foi o facto de a mensagem [Benguela] Stock decrementado ter chegado a aparecer no ecrã — mas, quando fomos verificar a base de dados, o stock estava intacto. Isto ensinou-nos que uma alteração feita dentro de uma transação ainda não confirmada é “provisória”: existe apenas para aquela sessão e desaparece com o `rollback`. A consistência foi mantida porque nenhum dos dois nós ficou com uma alteração isolada.

Também discutimos a principal limitação desta abordagem: trata-se de um 2PC (Two-Phase Commit) simplificado. Se o programa Python falhasse exatamente no intervalo entre o `commit()` de Benguela e o `commit()` de Luanda, um nó ficaria confirmado e o outro não, quebrando a atomicidade. Um 2PC verdadeiro resolve isto com uma fase de `prepare` coordenada por um gestor de transações, algo que está fora do âmbito deste laboratório mas que percebemos ser essencial em produção.

Sobre a Fase 0 e 1 (ambiente e schema)

A maior lição das fases iniciais foi perceber que um contentor “healthy” não significa um contentor “pronto e correto”. O healthcheck do MySQL apenas confirma que o servidor responde, não que a password está certa nem que os dados foram carregados. Aprendemos também o papel dos volumes Docker: enquanto o volume antigo existir, a inicialização (`MYSQL_ROOT_PASSWORD` e os scripts de `/docker-entrypoint-initdb.d`) não volta a correr. Daí a importância do `down -v` quando se quer um arranque verdadeiramente limpo.

4. Registo de Uso de IA

Ferramenta utilizada: Claude Code (modelo Claude Opus 4.8), via extensão no VS Code. Para cada interação documentamos o pedido feito, o que a IA respondeu e o que o grupo alterou ou validou.

#	Prompt do grupo (resumo)	Resposta da IA (resumo)	O que o grupo alterou/validou
1	Porque me dá ERROR 1045 Access denied ao ligar ao MySQL com -pkwanza2024?	Diagnosticou que o mysqladmin ping do healthcheck devolve exit 0 mesmo com password errada (logo “healthy” não prova nada) e que o volume antigo manteve a password original (MYSQL_ROOT_PASSWORD só se aplica na 1.ª inicialização). Sugeriu docker compose down -v && up -d.	Validámos correndo o ping com password errada (deu exit 0). Recriámos os volumes; a password passou a funcionar.
2	Como ligo o MySQL Workbench às bases nos contentores?	Indicou host 127.0.0.1, portas 3301/3307/3308, user root, e o aviso de não usar localhost em Linux (socket).	Usámos os parâmetros indicados para criar as 3 ligações no Workbench.
3	Porque dá erro Could not process parameters: str(...) no transacao.py?	Identificou uma vírgula entre as duas strings do INSERT INTO ITEM_VENDA, que fazia o execute() interpretar a 2.ª string como os parâmetros. Corrigiu para concatenação de strings adjacentes.	Aceitámos a correção e voltámos a correr o script com sucesso.
4	Executar a transação, forçar erro e registar resultados (passos 21-23).	Correu o script (sucesso e erro forçado), capturou os COUNT antes/depois e preencheu a tabela FASE-4B.	Revimos os valores e confirmámos a consistência após o rollback.
5	Gerar 50 produtos angolanos realistas e medir a propagação da replicação (P1-A).	Gerou o dataset dados/mercadokwanza_p2.sql (50 produtos, 9 categorias) e as views de fragmentação, correu no Master e mediu a propagação e o Seconds_Behind_Source.	Revimos a lista de produtos antes de inserir; validámos os COUNT nos 3 nós.
6	Escrever o script de migração para MongoDB.	Gerou scripts/migracao.py (PRODUTO → coleção produtos; VENDA + ITEM_VENDA → coleção vendas com itens embebidos).	Executámos e confirmámos no MongoDB as contagens (253 produtos, 30 002 vendas).

Todas as sugestões da IA foram revistas e validadas pelo grupo contra a base de dados antes de serem aceites; nenhum resultado foi assumido sem confirmação. As reflexões escritas deste relatório foram redigidas pelos elementos do grupo, com as próprias palavras.

5. Parte 2 — Implementação (Cenário A: Replicação + Fragmentação)

5.1 Arquitetura

Quatro nós em contentores Docker (ver docker-compose.yml):

Nó	Contentor	Papel	Porta host	server-id
Luanda	no-luanda	Master (escrita)	3301	1
Benguela	no-benguela	Slave / Réplica	3307	2
Huambo	no-huambo	Slave / Réplica	3308	3
MongoDB	mongo-kwanza	NoSQL (parte separada)	27017	—

O Master tem `--log-bin=mysql-bin` e `--binlog-format=ROW`. Os Slaves estão ligados ao Master via replicação assíncrona, confirmado com `SHOW REPLICA STATUS`: `Replica_IO_Running=Yes` e `Replica_SQL_Running=Yes`.

5.2 Expansão do Catálogo (passo 25) · dados/mercadokwanza_p2.sql

Inserção de 50 produtos com nomes realistas angolanos, distribuídos por 9 categorias (Alimentação, Bebidas, Higiene, Limpeza, Electrónica, Vestuário, Papelaria, Casa, Construção). Exemplos: Fuba de Milho Boa Safra 5kg, Cerveja Cuca Lata 33cl Pack 6, Pano Samakaka 6 Jardas, Saco de Cimento Nova Cimangola 50kg. O ficheiro completo está no repositório. A inserção é feita apenas no Master; a replicação propaga automaticamente para os Slaves.

5.3 Fragmentação Horizontal de STOCK (passo 28)

```
CREATE OR REPLACE VIEW frag_stock_luanda AS
  SELECT s.* FROM STOCK s JOIN LOJA l ON s.loja_id=l.id
  WHERE l.provincia_id=1;

CREATE OR REPLACE VIEW frag_stock_benguela AS
  SELECT s.* FROM STOCK s JOIN LOJA l ON s.loja_id=l.id
  WHERE l.provincia_id=2;
```

Fragmentação horizontal: cada fragmento contém as linhas de STOCK das lojas de uma província.

6. Parte 2 — Análise

Registo de Resultados — P1-A

Item	Valor
COUNT PRODUTO no Luanda antes dos INSERTs	203
COUNT PRODUTO no Benguela antes dos INSERTs	203
COUNT PRODUTO no Benguela imediatamente após INSERT	253 (≈ 119 ms depois)
COUNT PRODUTO no Benguela após 5 segundos	253
Seconds_Behind_Master máximo observado	0
Stock total fragmento Luanda	262 386
Stock total fragmento Benguela	248 892

Timestamps medidos

- T0 (instante do INSERT no Master): **21:15:40.857**
- T imediato (COUNT no Benguela): **21:15:40.976** → já mostrava 253
- T+5s: **21:16:18** → 253 (estável)

Evidência (SHOW REPLICA STATUS)

```

Replica_IO_Running: Yes
Replica_SQL_Running: Yes
Seconds_Behind_Source: 0
Last_Error:

```

Análise à Luz do Teorema CAP

O Teorema CAP afirma que, perante uma partição de rede (P), um sistema distribuído tem de escolher entre Consistência (C) e Disponibilidade (A).

- A replicação Master-Slave assíncrona do MySQL privilegia **Disponibilidade (AP)**: os Slaves continuam a responder a leituras mesmo que o Master fique inacessível, mas podem servir dados ligeiramente desatualizados (consistência eventual). No nosso teste o atraso foi ~0, mas sob carga elevada ou rede degradada o Seconds_Behind_Master aumentaria.
- A transação 2PC da Fase 4-B, pelo contrário, privilegia **Consistência (CP)**: exige que ambos os nós confirmem antes de validar; se um falhar, tudo é revertido — ao custo de bloquear/abortar (menor disponibilidade).
- **Conclusão**: o MercadoKwanza usa ambas as estratégias conforme a necessidade — replicação AP para o catálogo (leituras rápidas, tolerância a atraso) e transações CP para operações críticas de stock/venda (não pode haver dinheiro/stock inconsistente).

7. Parte 2 — Reflexões (individuais)

1. O Slave recebeu os dados antes ou depois dos 5 segundos? O que influencia este tempo de propagação?

No nosso teste, o Slave recebeu os 50 produtos quase imediatamente: medimos uma diferença de apenas cerca de 119 milésimos de segundo entre o instante do INSERT no Master e o momento em que o COUNT em Benguela já mostrava 253 produtos. Ou seja, a propagação aconteceu muito antes dos 5 segundos — quando voltámos a verificar aos 5 segundos, o valor já era exatamente o mesmo, e o `Seconds_Behind_Master` manteve-se sempre em 0.

Percebemos que este tempo de propagação não é fixo e depende de vários fatores. Os principais são: o volume e o tamanho das transações inseridas (50 linhas é muito pouco), a latência da rede entre os nós (aqui é uma rede Docker local, praticamente sem atraso), a carga de trabalho que o Master tem no momento, o formato do binlog (usámos ROW) e, sobretudo, a rapidez com que a thread SQL do Slave consegue ler e aplicar o relay log. Num cenário real, com servidores em províncias diferentes, ligações de internet mais lentas e milhares de operações por segundo, este atraso seria seguramente maior e mais visível.

2. Se neste momento o Master caísse, os Slaves teriam todos os dados? O que poderia ter ficado por replicar?

Não necessariamente. Como a replicação que usámos é assíncrona, o Master confirma a escrita ao cliente sem esperar que os Slaves a tenham recebido. Isto significa que, se o Master falhasse de forma abrupta, qualquer transação que já tivesse sido confirmada no Master mas ainda não tivesse sido transferida e aplicada pelos Slaves ficaria perdida. O tamanho desta “janela de perda” corresponde, na prática, ao valor de `Seconds_Behind_Master` no momento da falha.

No nosso caso concreto, como o atraso era 0, a perda seria mínima ou nula. No entanto, percebemos que isto foi sorte do cenário (pouca carga e rede local) e não uma garantia. Para reduzir este risco existiria a replicação semi-síncrona, em que o Master só confirma a escrita depois de pelo menos um Slave acusar a receção dos dados — mais seguro, mas mais lento.

3. A fragmentação de STOCK por província faz sentido para o negócio do MercadoKwanza? Que consultas seriam mais rápidas e quais seriam mais lentas?

Sim, faz sentido, porque o stock é, por natureza, um dado geograficamente local: cada província gere as suas próprias lojas e o seu próprio inventário. Ao fragmentar a tabela STOCK por `provincia_id`, cada região passa a trabalhar essencialmente sobre o seu fragmento, o que reflete bem a forma como o negócio funciona na realidade.

As consultas que ficariam mais rápidas são as locais — por exemplo, “qual o stock disponível nas lojas de Luanda” — porque cada fragmento contém apenas as linhas daquela província e o motor lê muito menos dados. Em contrapartida, as consultas que ficariam mais lentas são as globais, que precisam de olhar para todo o país — por exemplo, “qual o stock total do produto X em Angola inteira” — porque obrigam a juntar (com UNION ou agregação) os resultados de todos os fragmentos. Concluímos, por

isso, que a fragmentação é uma boa decisão se a maioria das consultas do dia a dia for regional, que é exatamente o caso de uma cadeia de supermercados como o MercadoKwanza.

8. Dificuldades

1. Access denied no MySQL apesar da password correta no compose

Diagnóstico: o healthcheck com mysqladmin ping ficava “healthy” mesmo com password errada (devolve exit 0 se o servidor responder), e o volume Docker antigo tinha mantido a password da primeira inicialização. Resolução: docker compose down -v para recriar os volumes do zero.

2. Could not process parameters: str(...) no transacao.py

Diagnóstico: uma vírgula a separar as duas strings do INSERT INTO ITEM_VENDA fazia o conector tratar a 2.^a string como os parâmetros. Resolução: remover a vírgula (strings adjacentes concatenam-se em Python).

9. Conclusão

Neste laboratório montámos um sistema de base de dados distribuído real, com um servidor principal em Luanda e duas réplicas em Benguela e Huambo. Aprendemos, na prática, que replicar dados não é instantâneo nem garantido: funciona muito bem quando tudo está saudável (a cópia chegou em milésimas de segundo), mas se o servidor principal falhar no momento errado pode perder-se informação.

Também percebemos que uma transação que mexe em dois sítios ao mesmo tempo tem de ser “tudo ou nada” — e vimos isso acontecer quando forçámos um erro e tudo foi revertido. Por fim, dividir o stock por província (fragmentação) torna as consultas locais mais rápidas, mas as consultas que olham para o país inteiro ficam mais pesadas. No fundo, não há solução perfeita: escolhe-se entre rapidez/disponibilidade e consistência consoante o que o negócio precisa.

10. GitHub

Repositório: <https://github.com/nunoom/bdd2-lab02-grupo1>

O repositório (público) inclui: docker-compose.yml, os datasets em dados/ (mercadokwanza_p1.sql e mercadokwanza_p2.sql), os scripts/ (transacao.py, transacao_erro.py, migracao.py, replicacao.sql), o relatório em relatorio/ e um README.md explicativo com a arquitetura, as portas de cada nó e instruções de execução.

Estrutura do repositório

```
bdd2-lab02-grupo1/
├── README.md      ← descrição, execução, conclusões
├── dados/         ← mercadokwanza_p1.sql, mercadokwanza_p2.sql
├── scripts/       ← transacao.py, migracao.py, replicacao.sql
├── docker-compose.yml ← configuração do cluster (4 nós)
└── relatorio/     ← Lab02_Grupo1.pdf
```

11. Referências (APA 7.ª edição)

- Brewer, E. (2012). CAP twelve years later: How the “rules” have changed. *Computer*, 45(2), 23–29. <https://doi.org/10.1109/MC.2012.37>
- Gilbert, S., & Lynch, N. (2002). Brewer’s conjecture and the feasibility of consistent, available, partition-tolerant web services. *ACM SIGACT News*, 33(2), 51–59. <https://doi.org/10.1145/564585.564601>
- Gray, J., & Lamport, L. (2006). Consensus on transaction commit. *ACM Transactions on Database Systems*, 31(1), 133–160. <https://doi.org/10.1145/1132863.1132867>
- Özsu, M. T., & Valduriez, P. (2020). *Principles of distributed database systems* (4th ed.). Springer. <https://doi.org/10.1007/978-3-030-26253-2>
- Silberschatz, A., Korth, H. F., & Sudarshan, S. (2020). *Database system concepts* (7th ed.). McGraw-Hill.
- Tanenbaum, A. S., & van Steen, M. (2017). *Distributed systems* (3rd ed.). Maarten van Steen.
- Oracle Corporation. (2024). *MySQL 8.0 reference manual — Replication*. <https://dev.mysql.com/doc/refman/8.0/en/replication.html>

“Um sistema distribuído é aquele em que a falha de um computador que você nem sabia que existia pode tornar o seu computador inutilizável.”

— Leslie Lamport